



ELC 3080 SPRING 2025

COMPUTER NETWORKS PROJECT

VIRTUAL NETWORKS LAB USING LINUX AND THE CORE
NETWORK EMULATOR



	sec	BN
Yahia Ebrahim	4	30
Youssef Khaled Abdel Raouf	4	37

JUNE 4, 2025
DR.RANIA OSAMA
DR.khaled Fouad

Contents

Table of figures	1
Question 2.i	3
Question 2.ii	4
Question 2.iii	5
Question 3.i	6
Question 4.i	8
Question 4.ii	8
Question 4.iii	8
Question 5.i	9
Question 5.ii	9
Question 6.i	11
Question 6.ii	12
Question 6.iii	13

Table of figures

Figure 1: window size 1Kb	3
Figure 2: window size 2Kb	3
Figure 3: window size 3Kb	3
Figure 4: window size 4Kb	3
Figure 5: window size 5Kb	3
Figure 6: window size 6Kb	3
Figure 7: window size 12kb	3
Figure 8: window size 16Kb	3
Figure 9: window size 24Kb	3
Figure 10: window size 36Kb	3
Figure 11: throughput vs window size	4
Figure 12: avg num of retransmission vs window size	4
Figure 13: TCP header in data segment	4
Figure 14: IP header in data segment	5
Figure 15: TCP header in ACK pkt	5
Figure 16: IP header in ACK pkt	5
Figure 17: throughput to n11	6
Figure 18: throughput to n7	6
Figure 19: link between n4 & n5 has capacity 10Mbps with zero loss both directions	7
Figure 20: link between n4 & n5 has capacity 3Mbps with zero loss both directions	7
Figure 21: link between n4 & n5 has capacity 10Mbps with 5% loss both directions	7
Figure 22: link between n4 & n5 has capacity 100Mbps with 10% loss both directions	7

Figure 23:link between n4 & n5 has capacity 10Mbps with 1% loss in directions from n4 to n5 and 0 loss in other direction	7
Figure 24:link between n4 & n5 has capacity 10Mbps with 0% loss in directions from n4 to n5 and 1% loss in other direction	7
Figure 25: Path before changing cost.....	9
Figure 26: Path after changing cost	9
Figure 27: Path from n7 to n11 after changing cost.....	9
Figure 28: Path from n7 to n11 before changing cost.....	9
Figure 29: Path from n11 to n7 after changing cost.....	10
Figure 30: Path from n11 to n7 before changing cost.....	10
Figure 31: database	11
Figure 32: routing table.....	11
Figure 33: First OSPF update message.....	12
Figure 34: Last ACK.....	12
Figure 35: new database.....	13
Figure 36: new routing table	13

Effect of TCP Window Size

Question 2.i

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 1k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35330 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  2.04 MBytes  1.71 Mbits/sec  0    5.45 KBytes
[ 4] 10.00-20.00 sec  2.85 MBytes  2.39 Mbits/sec  0    5.45 KBytes
[ 4] 20.00-30.00 sec  3.07 MBytes  2.57 Mbits/sec  0    5.45 KBytes
[ 4] 30.00-40.00 sec  3.18 MBytes  2.67 Mbits/sec  0    5.45 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  11.1 MBytes  2.34 Mbits/sec  0
[ 4] 0.00-40.00 sec  11.1 MBytes  2.34 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 1: window size 1Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 2k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35332 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  7.02 MBytes  5.89 Mbits/sec  0    7.07 KBytes
[ 4] 10.00-20.00 sec  7.94 MBytes  6.66 Mbits/sec  0    7.07 KBytes
[ 4] 20.00-30.00 sec  5.92 MBytes  4.97 Mbits/sec  0    7.07 KBytes
[ 4] 30.00-40.00 sec  5.87 MBytes  4.93 Mbits/sec  0    7.07 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  26.8 MBytes  5.61 Mbits/sec  0
[ 4] 0.00-40.00 sec  26.8 MBytes  5.61 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 2: window size 2Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 3k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35334 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  8.60 MBytes  7.22 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  7.35 MBytes  6.17 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  7.40 MBytes  6.21 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  9.24 MBytes  7.75 Mbits/sec  0    14.1 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  32.6 MBytes  6.84 Mbits/sec  0
[ 4] 0.00-40.00 sec  32.6 MBytes  6.84 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 3: window size 3Kb

```

root@n7:/tmp/pycore.54125/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 54566 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  8.24 MBytes  6.91 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  10.1 MBytes  8.50 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  7.54 MBytes  6.33 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  6.03 MBytes  5.06 Mbits/sec  0    14.1 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  31.9 MBytes  6.70 Mbits/sec  0
[ 4] 0.00-40.00 sec  31.9 MBytes  6.70 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.54125/n7.conf#

```

Figure 5: window size 5Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 6k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35342 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  2.50 MBytes  2.10 Mbits/sec  433  4.24 KBytes
[ 4] 10.00-20.00 sec  3.62 MBytes  3.04 Mbits/sec  656  4.24 KBytes
[ 4] 20.00-30.00 sec  3.63 MBytes  3.05 Mbits/sec  660  4.24 KBytes
[ 4] 30.00-40.00 sec  3.67 MBytes  3.08 Mbits/sec  662  4.24 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  13.4 MBytes  2.82 Mbits/sec  2411
[ 4] 0.00-40.00 sec  13.4 MBytes  2.81 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 7: window size 12Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 16k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35346 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  329 KBytes  270 Kbits/sec  121  7.07 KBytes
[ 4] 10.00-20.00 sec  297 KBytes  243 Kbits/sec  119  5.66 KBytes
[ 4] 20.00-30.00 sec  297 KBytes  243 Kbits/sec  120  5.66 KBytes
[ 4] 30.00-40.00 sec  277 KBytes  227 Kbits/sec  114  8.48 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  1.17 MBytes  246 Kbits/sec  474
[ 4] 0.00-40.00 sec  1.13 MBytes  238 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 9: window size 24Kb

```

root@n7:/tmp/pycore.54125/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 32k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 54562 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.01 sec  8.77 MBytes  7.35 Mbits/sec  0    14.1 KBytes
[ 4] 10.01-20.00 sec  9.06 MBytes  7.60 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  7.07 MBytes  5.93 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  7.46 MBytes  6.26 Mbits/sec  0    14.1 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  32.4 MBytes  6.79 Mbits/sec  0
[ 4] 0.00-40.00 sec  32.4 MBytes  6.78 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.54125/n7.conf#

```

Figure 4: window size 4Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 6k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35340 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  154 KBytes  126 Kbits/sec  87  4.24 KBytes
[ 4] 10.00-20.00 sec  148 KBytes  122 Kbits/sec  89  4.24 KBytes
[ 4] 20.00-30.00 sec  148 KBytes  122 Kbits/sec  90  4.24 KBytes
[ 4] 30.00-40.00 sec  143 KBytes  117 Kbits/sec  88  4.24 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  584 KBytes  122 Kbits/sec  354
[ 4] 0.00-40.00 sec  584 KBytes  120 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 6: window size 6Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 16k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35344 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  447 KBytes  366 Kbits/sec  137  7.07 KBytes
[ 4] 10.00-20.00 sec  495 KBytes  405 Kbits/sec  139  5.66 KBytes
[ 4] 20.00-30.00 sec  417 KBytes  342 Kbits/sec  126  8.48 KBytes
[ 4] 30.00-40.00 sec  417 KBytes  342 Kbits/sec  134  4.24 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  1.73 MBytes  364 Kbits/sec  536
[ 4] 0.00-40.00 sec  1.71 MBytes  358 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 8: window size 16Kb

```

root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 32k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 35348 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  274 KBytes  225 Kbits/sec  111  7.07 KBytes
[ 4] 10.00-20.00 sec  161 KBytes  132 Kbits/sec  95  19.8 KBytes
[ 4] 20.00-30.00 sec  214 KBytes  175 Kbits/sec  114  7.07 KBytes
[ 4] 30.00-40.00 sec  189 KBytes  155 Kbits/sec  98  15.6 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  839 KBytes  172 Kbits/sec  418
[ 4] 0.00-40.00 sec  789 KBytes  162 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 10: window size 36Kb

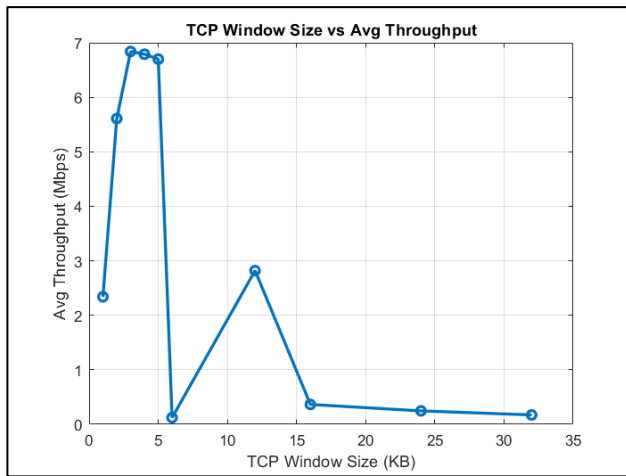


Figure 11: throughput vs window size

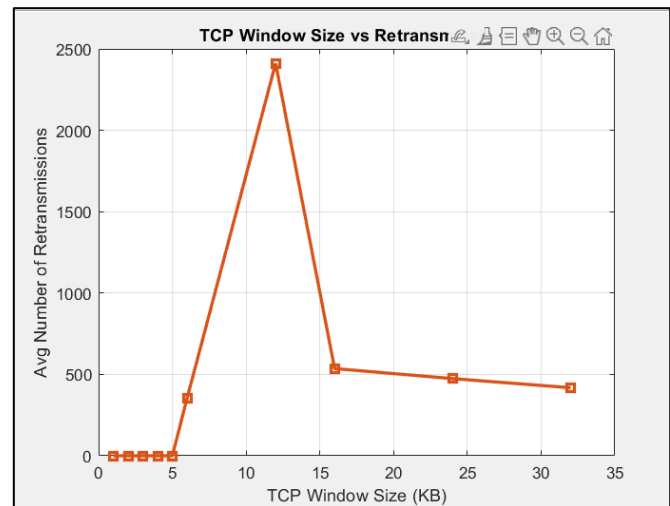


Figure 12: avg num of retransmission vs window size

The zigzag behavior is typically due to TCP congestion control and packet loss interactions:

- Larger window size means more data in flight, which can exceed the network's capacity.
- When congestion occurs:
 - Packet drops trigger TCP congestion control (e.g., fast retransmit, congestion avoidance).
 - TCP reduces its sending rate, leading to a drop in throughput.
- Depending on network timing and conditions, some window sizes may
 - Recover quickly or React too conservatively leading to lower throughput despite fewer retransmissions

Question 2.ii

```

Transmission Control Protocol, Src Port: 35329 (35329), Dst Port: targus-getdata1 (5201), Seq: 42, Ack: 2, Len: 71
  Source port: 35329 (35329)
  Destination port: targus-getdata1 (5201)
  [Stream index: 1]
  Sequence number: 42 (relative sequence number)
  [Next sequence number: 113 (relative sequence number)]
  Acknowledgement number: 2 (relative ack number)
  Header length: 32 bytes
  ▸ Flags: 0x018 (PSH, ACK)
  Window size value: 913
  [Calculated window size: 14608]
  [Window size scaling factor: 16]
  ▸ Checksum: 0x2d95 [validation disabled]
  ▾ Options: (12 bytes)
    No-Operation (NOP)
    No-Operation (NOP)
    ▸ Timestamps: TSval 4055805, TSecr 4055805
    ▸ [SEQ/ACK analysis]
  ▸ Data (71 bytes)

```

Figure 13: TCP header in data segment

```

Frame 80: 139 bytes on wire (1112 bits), 139 bytes captured (1112 bits)
Linux cooked capture
  Packet type: Unicast to another host (3)
  Link-layer address type: 1
  Link-layer address length: 6
  Source: 00:00:00_aa:00:18 (00:00:00:aa:00:18)
  Protocol: IP (0x0800)
Internet Protocol Version 4, Src: 10.0.12.20 (10.0.12.20), Dst: 10.0.13.20 (10.0.13.20)
  Version: 4
  Header length: 20 bytes
  Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00: Not-ECT (Not ECN-Capable Transport))
  Total Length: 123
  Identification: 0xf826 (63526)
  Flags: 0x02 (Don't Fragment)
  Fragment offset: 0
  Time to live: 64
  Protocol: TCP (6)
  Header checksum: 0x152f [correct]
  Source: 10.0.12.20 (10.0.12.20)
  Destination: 10.0.13.20 (10.0.13.20)

```

Figure 14: IP header in data segment

TCP header: 32 (20 + 12 options)

IP header: 20

Ethernet header: $((139-71)-20-32 = 16)$

Question 2.iii

```

Transmission Control Protocol, Src Port: targus-getdata1 (5201), Dst Port: 35329 (35329), Seq: 2, Ack: 113, Len: 0
  Source port: targus-getdata1 (5201)
  Destination port: 35329 (35329)
  [Stream index: 1]
  Sequence number: 2 (relative sequence number)
  Acknowledgement number: 113 (relative ack number)
  Header length: 32 bytes
  Flags: 0x010 (ACK)
  Window size value: 905
  [Calculated window size: 14480]
  [Window size scaling factor: 16]
  Checksum: 0x2d4e [validation disabled]
  Options: (12 bytes)
    No-Operation (NOP)
    No-Operation (NOP)
    Timestamps: TSval 4055805, TSecr 4055805
      Kind: Timestamp (8)
      Length: 10
      Timestamp value: 4055805
      Timestamp echo reply: 4055805
  [SEQ/ACK analysis]

```

Figure 15: TCP header in ACK pkt

```

Frame 88: 68 bytes on wire (544 bits), 68 bytes captured (544 bits)
Linux cooked capture
  Packet type: Unicast to another host (3)
  Link-layer address type: 1
  Link-layer address length: 6
  Source: 00:00:00_aa:00:1a (00:00:00:aa:00:1a)
  Protocol: IP (0x0800)
Internet Protocol Version 4, Src: 10.0.13.20 (10.0.13.20), Dst: 10.0.12.20 (10.0.12.20)
  Version: 4
  Header length: 20 bytes
  Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00: Not-ECT (Not ECN-Capable Transport))
  Total Length: 52
  Identification: 0x147b (5243)
  Flags: 0x02 (Don't Fragment)
  Fragment offset: 0
  Time to live: 64
  Protocol: TCP (6)
  Header checksum: 0xf921 [correct]
  Source: 10.0.13.20 (10.0.13.20)
  Destination: 10.0.12.20 (10.0.12.20)

```

Figure 16: IP header in ACK pkt

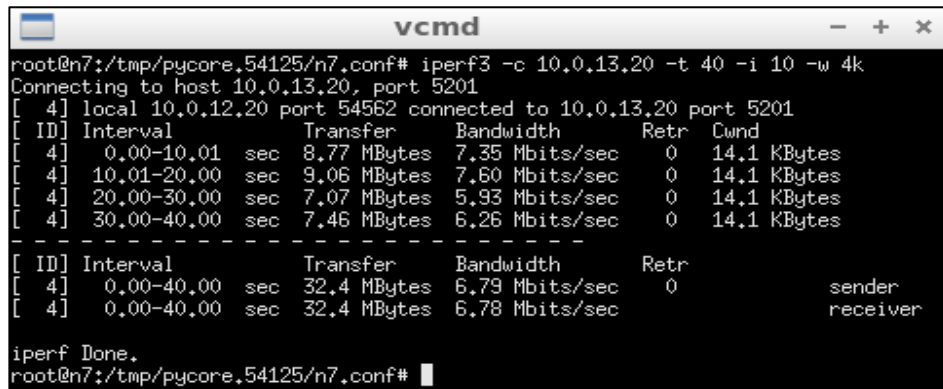
TCP header: (32 = 20 + 12)

IP header: (20)

Ethernet header: $(68 - 32 - 20 = 16)$

TCP short versus long paths

Question 3.i



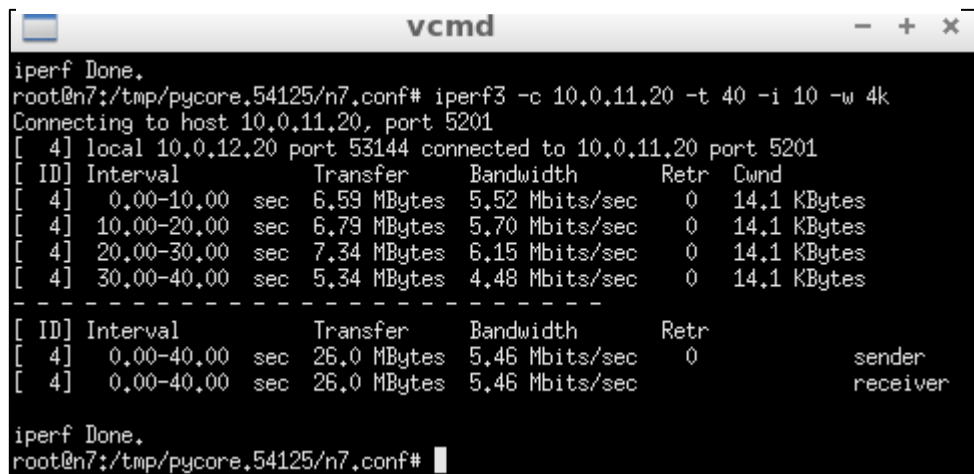
```

root@n7:/tmp/pycore.54125/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 54562 connected to 10.0.13.20 port 5201
[ ID] Interval           Transfer     Bandwidth   Retr  Cwnd
[ 4]  0.00-10.01  sec    8.77 MBytes  7.35 Mbits/sec    0   14.1 KBytes
[ 4] 10.01-20.00  sec    9.06 MBytes  7.60 Mbits/sec    0   14.1 KBytes
[ 4] 20.00-30.00  sec    7.07 MBytes  5.93 Mbits/sec    0   14.1 KBytes
[ 4] 30.00-40.00  sec    7.46 MBytes  6.26 Mbits/sec    0   14.1 KBytes
-----
[ ID] Interval           Transfer     Bandwidth   Retr
[ 4]  0.00-40.00  sec   32.4 MBytes  6.79 Mbits/sec    0             sender
[ 4]  0.00-40.00  sec   32.4 MBytes  6.78 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.54125/n7.conf#

```

Figure 17: throughput to n11



```

iperf Done.
root@n7:/tmp/pycore.54125/n7.conf# iperf3 -c 10.0.11.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.11.20, port 5201
[ 4] local 10.0.12.20 port 53144 connected to 10.0.11.20 port 5201
[ ID] Interval           Transfer     Bandwidth   Retr  Cwnd
[ 4]  0.00-10.00  sec    6.59 MBytes  5.52 Mbits/sec    0   14.1 KBytes
[ 4] 10.00-20.00  sec    6.79 MBytes  5.70 Mbits/sec    0   14.1 KBytes
[ 4] 20.00-30.00  sec    7.34 MBytes  6.15 Mbits/sec    0   14.1 KBytes
[ 4] 30.00-40.00  sec    5.34 MBytes  4.48 Mbits/sec    0   14.1 KBytes
-----
[ ID] Interval           Transfer     Bandwidth   Retr
[ 4]  0.00-40.00  sec   26.0 MBytes  5.46 Mbits/sec    0             sender
[ 4]  0.00-40.00  sec   26.0 MBytes  5.46 Mbits/sec                  receiver

iperf Done.
root@n7:/tmp/pycore.54125/n7.conf#

```

Figure 18: throughput to n7

Throughput (to n11) = 7.91 Mbps

Throughput (to n7) = 5.46 Mbps

Why throughput drops?

- Path Length (Hop Count) Difference
 - Path to n11 (via n2): ~2 hops.
 - Path to n8 (via n5 → n1 → n2): ~3+ hops.
 - More hops = more routers processing packets = higher end-to-end latency and increased chance of congestion.
 - Each node introduces buffering and possible queuing delay.
 - More intermediate nodes = higher probability of delays or buffer drops.
 - Even if each link has the same bandwidth (10 Mbps), their latencies accumulate.
- TCP Congestion Control & RTT
 - TCP's performance is heavily influenced by RTT (Round Trip Time) and congestion window.
 - TCP uses ACKs to grow the congestion window. More hops = higher RTT = slower window growth, especially with a small fixed window (4K).

Higher Link Capacity with Drops versus Reliable Lower Capacity

```

vcmd
[ 4] 10,00-20,00 sec 7.96 MBytes 6.68 Mbits/sec 0 14.1 KBytes
[ 4] 20,00-30,00 sec 5.69 MBytes 4.78 Mbits/sec 0 14.1 KBytes
[ 4] 30,00-40,00 sec 8.12 MBytes 6.74 Mbits/sec 0 14.1 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,10 sec 29.8 MBytes 6.24 Mbits/sec 0
[ 4] 0,00-40,10 sec 29.8 MBytes 6.24 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0,13,20 -t 40 -i 10 -w 4k
Connecting to host 10.0,13,20, port 5201
[ 4] local 10.0,12,20 port 35352 connected to 10.0,13,20 port 5201
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0,00-10,00 sec 10.5 MBytes 8.83 Mbits/sec 0 14.1 KBytes
[ 4] 10,00-20,00 sec 8.51 MBytes 7.14 Mbits/sec 0 14.1 KBytes
[ 4] 20,00-30,00 sec 9.11 MBytes 7.64 Mbits/sec 0 14.1 KBytes
[ 4] 30,00-40,00 sec 9.59 MBytes 8.05 Mbits/sec 0 14.1 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 37.7 MBytes 7.91 Mbits/sec 0
[ 4] 0,00-40,00 sec 37.7 MBytes 7.91 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 19: link between n4 & n5 has capacity 10Mbps with zero loss both directions

```

vcmd
[ 4] 10,00-20,00 sec 8.51 MBytes 7.14 Mbits/sec 0 14.1 KBytes
[ 4] 20,00-30,00 sec 9.11 MBytes 7.64 Mbits/sec 0 14.1 KBytes
[ 4] 30,00-40,00 sec 9.59 MBytes 8.05 Mbits/sec 0 14.1 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 37.7 MBytes 7.91 Mbits/sec 0
[ 4] 0,00-40,00 sec 37.7 MBytes 7.91 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0,13,20 -t 40 -i 10 -w 4k
Connecting to host 10.0,13,20, port 5201
[ 4] local 10.0,12,20 port 35354 connected to 10.0,13,20 port 5201
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0,00-10,00 sec 3.41 MBytes 2.86 Mbits/sec 0 14.1 KBytes
[ 4] 10,00-20,00 sec 3.35 MBytes 2.81 Mbits/sec 0 14.1 KBytes
[ 4] 20,00-30,00 sec 3.37 MBytes 2.83 Mbits/sec 0 14.1 KBytes
[ 4] 30,00-40,00 sec 3.31 MBytes 2.78 Mbits/sec 0 8.48 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 13.4 MBytes 2.82 Mbits/sec 0
[ 4] 0,00-40,00 sec 13.4 MBytes 2.82 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 20: link between n4 & n5 has capacity 3Mbps with zero loss both directions

```

vcmd
[ 4] 10,00-20,00 sec 3.35 MBytes 2.81 Mbits/sec 0 14.1 KBytes
[ 4] 20,00-30,00 sec 3.37 MBytes 2.83 Mbits/sec 0 14.1 KBytes
[ 4] 30,00-40,00 sec 3.31 MBytes 2.78 Mbits/sec 0 8.48 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 13.4 MBytes 2.82 Mbits/sec 0
[ 4] 0,00-40,00 sec 13.4 MBytes 2.82 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0,13,20 -t 40 -i 10 -w 4k
Connecting to host 10.0,13,20, port 5201
[ 4] local 10.0,12,20 port 35356 connected to 10.0,13,20 port 5201
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0,00-10,00 sec 806 KBytes 660 Kbits/sec 29 2.83 KBytes
[ 4] 10,00-20,00 sec 837 KBytes 686 Kbits/sec 38 4.24 KBytes
[ 4] 20,00-30,00 sec 752 KBytes 616 Kbits/sec 38 4.24 KBytes
[ 4] 30,00-40,00 sec 771 KBytes 631 Kbits/sec 41 4.24 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 3.09 MBytes 648 Kbits/sec 146
[ 4] 0,00-40,00 sec 3.09 MBytes 648 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 21: link between n4 & n5 has capacity 10Mbps with 5% loss both directions

```

vcmd
[ 4] 10,00-20,00 sec 837 KBytes 686 Kbits/sec 38 4.24 KBytes
[ 4] 20,00-30,00 sec 752 KBytes 616 Kbits/sec 38 4.24 KBytes
[ 4] 30,00-40,00 sec 771 KBytes 631 Kbits/sec 41 4.24 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 3.09 MBytes 648 Kbits/sec 146
[ 4] 0,00-40,00 sec 3.09 MBytes 648 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0,13,20 -t 40 -i 10 -w 4k
Connecting to host 10.0,13,20, port 5201
[ 4] local 10.0,12,20 port 35358 connected to 10.0,13,20 port 5201
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0,00-10,00 sec 184 KBytes 151 Kbits/sec 21 4.24 KBytes
[ 4] 10,00-20,00 sec 160 KBytes 131 Kbits/sec 19 4.24 KBytes
[ 4] 20,00-30,00 sec 441 KBytes 361 Kbits/sec 37 4.24 KBytes
[ 4] 30,00-40,00 sec 153 KBytes 125 Kbits/sec 15 2.83 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 938 KBytes 192 Kbits/sec 92
[ 4] 0,00-40,00 sec 933 KBytes 191 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 22: link between n4 & n5 has capacity 100Mbps with 10% loss both directions

```

vcmd
[ 4] 10,00-20,00 sec 160 KBytes 131 Kbits/sec 19 4.24 KBytes
[ 4] 20,00-30,00 sec 441 KBytes 361 Kbits/sec 37 4.24 KBytes
[ 4] 30,00-40,00 sec 153 KBytes 125 Kbits/sec 15 2.83 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 938 KBytes 192 Kbits/sec 92
[ 4] 0,00-40,00 sec 933 KBytes 191 Kbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0,13,20 -t 40 -i 10 -w 4k
Connecting to host 10.0,13,20, port 5201
[ 4] local 10.0,12,20 port 35360 connected to 10.0,13,20 port 5201
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0,00-10,00 sec 6.72 MBytes 5.64 Mbits/sec 12 8.48 KBytes
[ 4] 10,00-20,00 sec 6.19 MBytes 5.20 Mbits/sec 11 8.48 KBytes
[ 4] 20,00-30,00 sec 8.44 MBytes 7.08 Mbits/sec 8 8.48 KBytes
[ 4] 30,00-40,00 sec 6.38 MBytes 5.35 Mbits/sec 14 8.48 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 27.7 MBytes 5.82 Mbits/sec 45
[ 4] 0,00-40,00 sec 27.7 MBytes 5.82 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 23: link between n4 & n5 has capacity 10Mbps with 1% loss in directions from n4 to n5 and 0 loss in other direction

```

vcmd
[ 4] 10,00-20,00 sec 6.19 MBytes 5.20 Mbits/sec 11 8.48 KBytes
[ 4] 20,00-30,00 sec 8.44 MBytes 7.08 Mbits/sec 8 8.48 KBytes
[ 4] 30,00-40,00 sec 6.38 MBytes 5.35 Mbits/sec 14 8.48 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 27.7 MBytes 5.82 Mbits/sec 45
[ 4] 0,00-40,00 sec 27.7 MBytes 5.82 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf# iperf3 -c 10.0,13,20 -t 40 -i 10 -w 4k
Connecting to host 10.0,13,20, port 5201
[ 4] local 10.0,12,20 port 35362 connected to 10.0,13,20 port 5201
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0,00-10,00 sec 4.55 MBytes 3.82 Mbits/sec 22 8.48 KBytes
[ 4] 10,00-20,00 sec 3.77 MBytes 3.16 Mbits/sec 26 8.48 KBytes
[ 4] 20,00-30,00 sec 3.18 MBytes 2.67 Mbits/sec 27 8.48 KBytes
[ 4] 30,00-40,00 sec 3.03 MBytes 2.54 Mbits/sec 23 5.66 KBytes

[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0,00-40,00 sec 14.5 MBytes 3.05 Mbits/sec 98
[ 4] 0,00-40,00 sec 14.5 MBytes 3.05 Mbits/sec

iperf Done.
root@n7:/tmp/pycore.52867/n7.conf#

```

Figure 24: link between n4 & n5 has capacity 10Mbps with 0% loss in directions from n4 to n5 and 1% loss in other direction

	a	b	c	d	e	f
Link capacity	10Mbps	3Mbps	10Mbps	100Mbps	10Mbps	10Mbps
Forward link loss	0	0	5%	10%	0	1%
Reverse link loss	0	0	5%	10%	1%	0
throughput	7.9Mbps	2.82Mbps	648Kbps	192Kbps	5.82Mbps	3.05Mbps

Question 4.i

Compare throughputs in cases a, b, c.

- (a) performs best due to 0% loss & 10Mbps link capacity. (throughput = 7.9 Mbps)
- (b) is bad — 0% loss but 3Mbps link capacity . (throughput = 2.82 Mbps)
- (c) is worst — 5% loss ruins TCP completely, despite high bandwidth. (throughput = 648 Kbps)

Why is (b) better than (c)?

TCP sacrifices speed for reliability. Loss causes congestion control to reduce the send rate — so even though (c) has more bandwidth, TCP can't use it effectively due to loss.

Question 4.ii

Compare throughputs in cases b, c and d.

- (b) performs best due to 0% loss & 3Mbps link capacity. (throughput = 2.82 Mbps)
- (c) is bad — 5% loss chokes TCP. (throughput = 648 kbps)
- (d) is worst — 10% loss ruins TCP completely, despite high bandwidth. (throughput = 192 kbps)

Which is better? Why?

B is better, for the same reason as TCP sacrifices speed for reliability. Loss causes congestion control to reduce the send rate — so even though (c&d) has more bandwidth, TCP can't use it effectively due to loss.

Question 4.iii

Compare throughputs in cases e and f?

- (e) is better, (e) link between n4 & n5 has capacity 10Mbps with 1% loss in directions from n4 to n5 and 0 loss in other direction. (throughput = 5.82 Mbps)
- (f) link between n4 & n5 has capacity 10Mbps with 0% loss in directions from n4 to n5 and 1% loss in other direction. (throughput = 3.05 Mbps)

Which is better? Why?

Case e: 10 Mbps, 1% loss from n4 → n5 (reverse path) affects ACKs.

TCP still suffers, but less than in case (f) because:

Lost ACKs = delayed RTT estimates and retransmits. But it doesn't reduce congestion window as aggressively.

Case f: 10 Mbps, 1% loss from n5 → n4 (Forward path) affects data packets.

TCP will suffer because the sender sees this loss and reduces its rate.

OSPF Link Cost Changes

Question 5.i

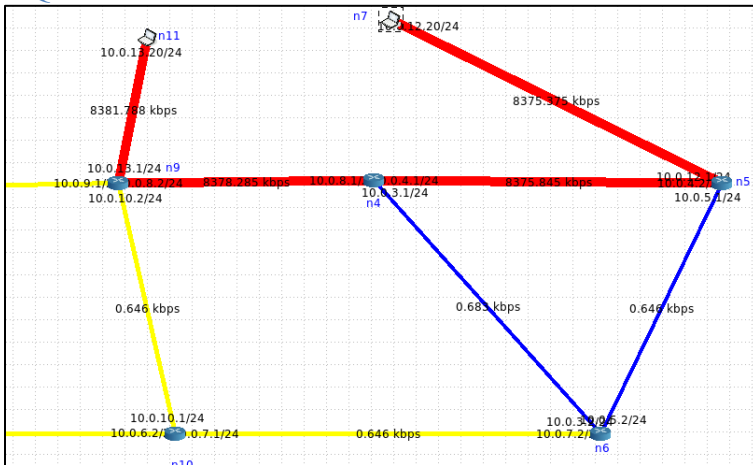


Figure 25: Path before changing cost.

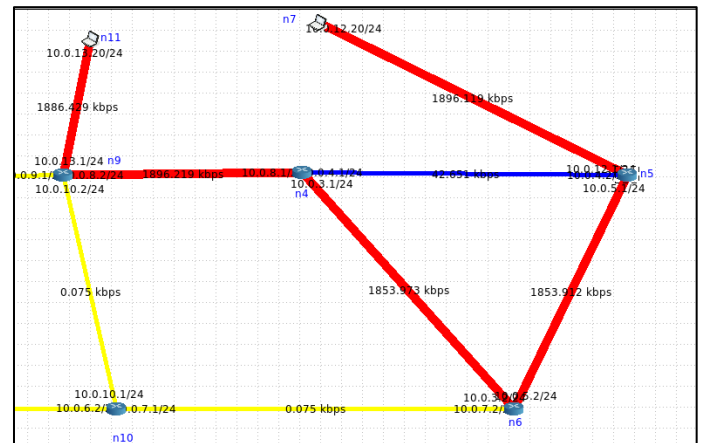


Figure 26: Path after changing cost

Path after step 3 is: $n7 \rightarrow n5 \rightarrow n4 \rightarrow n9 \rightarrow n11$.

Path after step6 is: $n7 \rightarrow n5 \rightarrow n6 \rightarrow n4 \rightarrow n9 \rightarrow n11$.

OSPF path changed from the direct link $n5 \rightarrow n4$ to an indirect path via $n6$ because:

- OSPF cost on the $n5 \rightarrow n4$ link was increased to 40.
- Now Old path cost is : $n7 \rightarrow n5$ (10) + $n5 \rightarrow n4$ (40) + $n4 \rightarrow n9$ (10) + $n9 \rightarrow n11$ (10) = **70** . while the new path cost is: $n7 \rightarrow n5$ (10) + $n5 \rightarrow n6$ (10) + $n6 \rightarrow n4$ (10) + $n4 \rightarrow n9$ (10) + $n9 \rightarrow n11$ (10) = **50** .
- OSPF always picks the route with the lowest total cost by rerunning Dijkstra's algorithm.

Question 5.ii

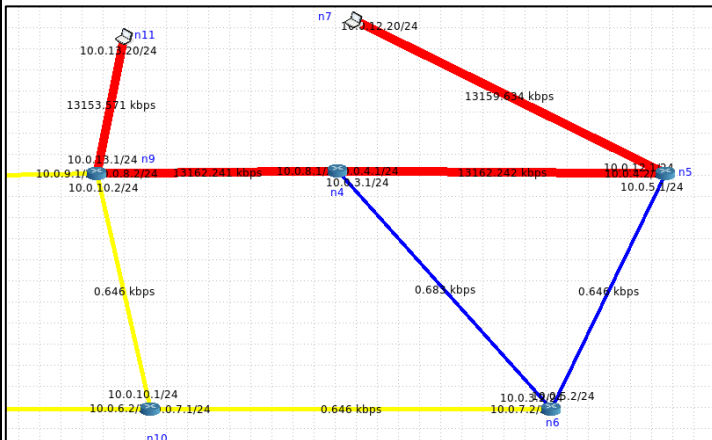


Figure 28: Path from n7 to n11 before changing cost.

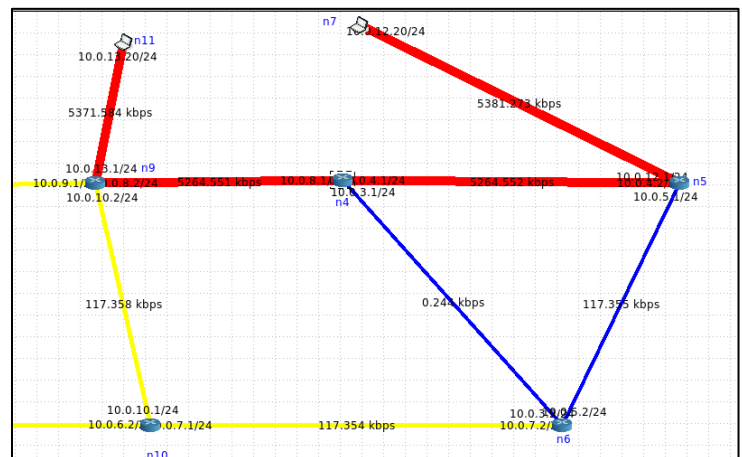


Figure 27: Path from n7 to n11 after changing cost.

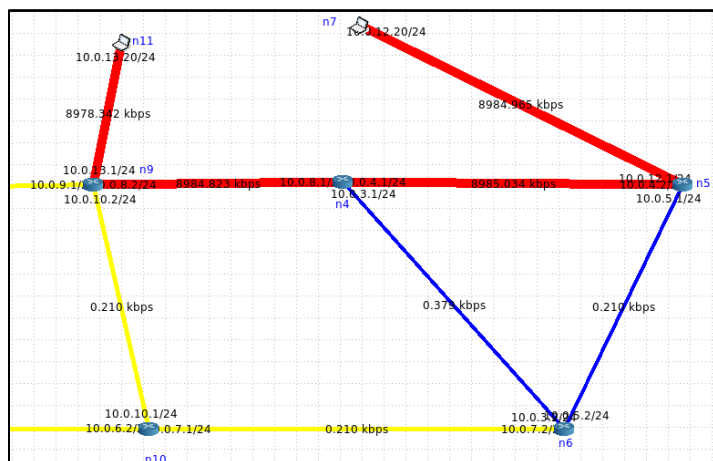


Figure 30: Path from n11 to n7 before changing cost.

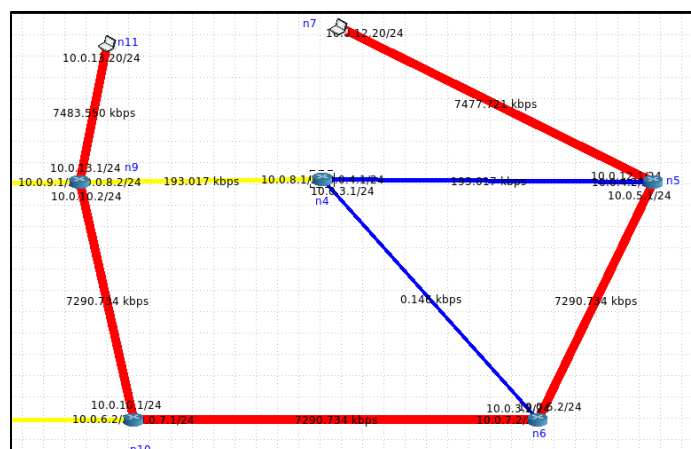


Figure 29: Path from n11 to n7 after changing cost.

For n7 to n11:

Before: $n7 \rightarrow n5 \rightarrow n4 \rightarrow n9 \rightarrow n11$.

After: doesn't change

For n11 to n7:

Before: $n11 \rightarrow n9 \rightarrow n4 \rightarrow n5 \rightarrow n7$.

After: $n11 \rightarrow n9 \rightarrow n10 \rightarrow n6 \rightarrow n5 \rightarrow n7$.

What happens in the paths of the two connections? Explain what happens. What do you conclude?

- The path from n7 to n11 did not change because OSPF still saw the path through $n5 \rightarrow n4 \rightarrow n9 \rightarrow n11$ as the lowest cost from n7's point of view because the link in the direction from n5 to n4 still had a low cost.
- The path from n11 to n7 changed because the increased cost on the n4 to n5 link made that old route has more cost from n11's perspective so OSPF recalculated and selected an alternative lower-cost path.

OSPF Database Updates

Question 6.i

```

vcmd
root@n2:/tmp/pycore.52867/n2.conf# vtysh
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

    OSPF Router with ID (10.0.1.2)

        Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum  Link count
10.0.0.1     10.0.0.1     1330 0x80000018 0xd6cb 3
10.0.0.2     10.0.0.2     1210 0x8000001a 0x7a18 3
10.0.1.2     10.0.1.2     1469 0x80000019 0x6f19 3
10.0.3.1     10.0.3.1     401  0x8000001a 0x273a 3
10.0.3.2     10.0.3.2     1482 0x80000019 0x8fed 3
10.0.5.1     10.0.5.1     1452 0x80000018 0x7415 3
10.0.6.2     10.0.6.2     1191 0x80000019 0xacbb 3
10.0.8.2     10.0.8.2     1280 0x8000001a 0x0623 4

        Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum
10.0.0.2     10.0.0.2     150  0x80000011 0x3fdb
10.0.1.2     10.0.1.2     879  0x80000010 0x46d1
10.0.2.1     10.0.1.2     939  0x80000010 0x33e5
10.0.3.2     10.0.3.2     212  0x80000011 0x4bc3
10.0.4.2     10.0.5.1     1202 0x80000010 0x46c6
10.0.5.1     10.0.5.1     812  0x80000010 0x4bc0
10.0.6.2     10.0.6.2     641  0x80000011 0x2bdc
10.0.7.1     10.0.6.2     771  0x80000010 0x3bca
10.0.8.2     10.0.8.2     1090 0x80000010 0x34cc
10.0.9.1     10.0.8.2     250  0x80000011 0x0df3
10.0.10.2    10.0.8.2     600  0x80000010 0x43b7
  
```

Figure 31: database

```

vcmd
n2# show ip ospf route
===== OSPF network routing table =====
N   10.0.0.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.2.2, eth1
N   10.0.1.0/24      [10] area: 0.0.0.0
    directly attached to eth0
N   10.0.2.0/24      [10] area: 0.0.0.0
    directly attached to eth1
N   10.0.3.0/24      [30] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.4.0/24      [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.5.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.6.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
N   10.0.7.0/24      [30] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.8.0/24      [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.9.0/24      [10] area: 0.0.0.0
    directly attached to eth2
N   10.0.10.0/24     [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.11.0/24     [20] area: 0.0.0.0
    via 10.0.2.2, eth1
N   10.0.12.0/24     [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.13.0/24     [20] area: 0.0.0.0
    via 10.0.9.1, eth2

===== OSPF router routing table =====
===== OSPF external routing table =====

n2#
  
```

Figure 32: routing table

Database:

➤ Router Link States (Area 0.0.0.0)

Each line here represents one router LSA. A router LSA describes the links (interfaces) a specific router has, their types, and the costs associated with them.

➤ Net Link States (Area 0.0.0.0)

Each line here represents one network LSA which is created by the designated router (DR) on a broadcast network (like Ethernet). It describes which routers are connected to the same shared network segment.

- Link ID: The ID of the router being described.
- ADV Router: The router that originated this LSA (same as Link ID in Router LSAs).
- Age: Time since the LSA was originated (in seconds).
- Seq#: Sequence number; updated when the LSA is changed.
- CkSum: A checksum used to detect corruption.
- Link count: How many links (interfaces) this router has.

Routing table:

It shows how the router will forward packets to reach different IP networks based on OSPF's shortest path calculations.

- **Network:** The destination subnet this router knows how to reach.

- **[Cost]:** The total OSPF cost (sum of link costs) to reach that network.
- **Via:** The next-hop router's IP address.
- **Interface:** Which interface to use to send packets toward the next hop.

Why some destinations have more than route in the routing tables?

OSPF allows multiple routes to the same destination if they have the same total cost so there can be multiple minimum cost paths to reach the same destination.

Question 6.ii

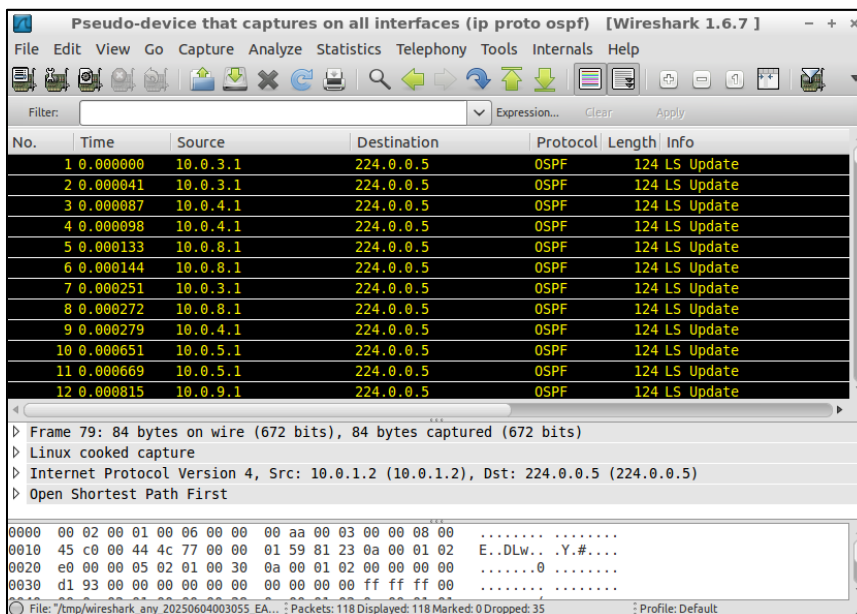


Figure 33: First OSPF update message.

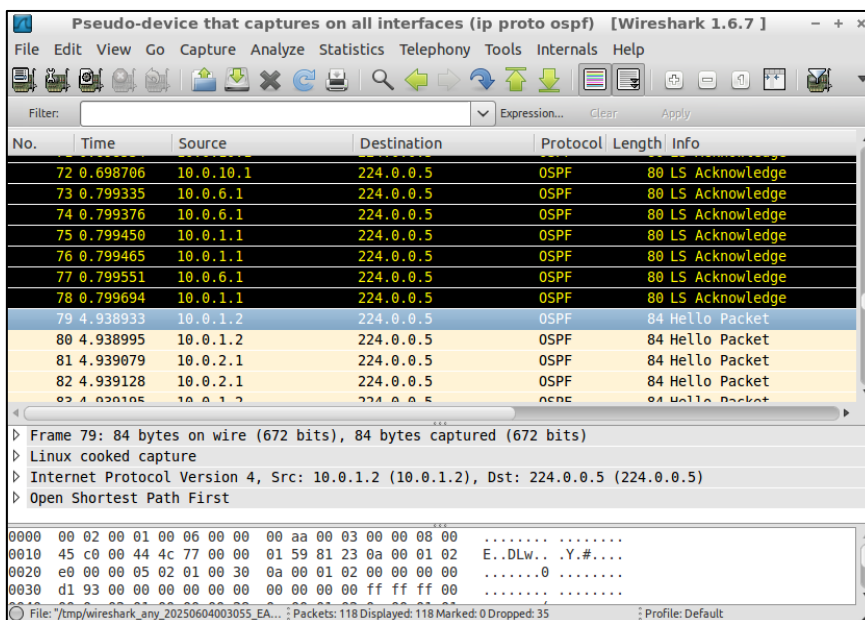


Figure 34: Last ACK.

how long it took the network to exchange link state packets and adjust routing tables ?

required time = time of first OSPF update packet – time of last ACK = 0.799694 – 0 = 0.7996449 sec

Question 6.iii

```

vcmd
n2# show ip ospf database

  OSPF Router with ID (10.0.1.2)

    Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum  Link count
10.0.0.1     10.0.0.1      82  0x8000001a 0xd2cd  3
10.0.0.2     10.0.0.2     1712 0x8000001b 0x7819  3
10.0.1.2     10.0.1.2      201 0x8000001b 0x6b1b  3
10.0.3.1     10.0.3.1       73 0x80000021 0x9452  1
10.0.3.2     10.0.3.2       44 0x8000001c 0xd2b7  3
10.0.5.1     10.0.5.1       35 0x8000001b 0xddb9  3
10.0.6.2     10.0.6.2     1643 0x8000001a 0xaaab  3
10.0.8.2     10.0.8.2       32 0x8000001c 0x0a32  4

    Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum
10.0.0.2     10.0.0.2     662 0x80000012 0x3ddc
10.0.1.2     10.0.1.2     1411 0x80000011 0x44d2
10.0.2.1     10.0.1.2     1391 0x80000011 0x31e6
10.0.3.2     10.0.3.2     3600 0x80000012 0x49c4
10.0.4.2     10.0.5.1     3600 0x80000011 0x44c7
10.0.5.1     10.0.5.1     1264 0x80000011 0x49c1
10.0.6.2     10.0.6.2     1133 0x80000012 0x29dd
10.0.7.1     10.0.6.2     1303 0x80000011 0x39cb
10.0.8.2     10.0.8.2     3600 0x80000011 0x32cd
10.0.9.1     10.0.8.2      732 0x80000012 0x0bf4
10.0.10.2    10.0.8.2     1082 0x80000011 0x41b8

```

Figure 35: new database

```

vcmd
n2# show ip ospf route
===== OSPF network routing table =====
N   10.0.0.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.2.2, eth1
N   10.0.1.0/24      [10] area: 0.0.0.0
    directly attached to eth0
N   10.0.2.0/24      [10] area: 0.0.0.0
    directly attached to eth1
N   10.0.3.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.4.0/24      [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.5.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.6.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.7.0/24      [30] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.8.0/24      [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.9.0/24      [10] area: 0.0.0.0
    directly attached to eth2
N   10.0.10.0/24     [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.11.0/24     [20] area: 0.0.0.0
    via 10.0.2.2, eth1
N   10.0.12.0/24     [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.13.0/24     [20] area: 0.0.0.0
    via 10.0.9.1, eth2

===== OSPF router routing table =====
===== OSPF external routing table =====
n2#

```

Figure 36: new routing table

Database: the LSA for router 10.0.4.1 (n4) is missing as it's no longer advertising because it's down.

Routing table: routes that passed through n4 are now either gone (if no alternative exists), or redirected via alternate routers, such as n6, n5, or n9. And OSPF recalculates paths based on the updated network topology.